

Fast Greedy Heuristics with Linear-Time Redundant Removal and Local Swap for the Minimum k -Dominating Set Problem on Large-Scale Networks

Tho Chi Luong*

International School,
Vietnam National University, Hanoi
Hanoi, Vietnam

May 14, 2026

Abstract

Given a graph $G = (V, E)$ and integer $k \geq 1$, a k -dominating set is a vertex subset $D \subseteq V$ inside whose distance- k neighbourhood every vertex of G lies. The minimum-cardinality version (MkDSP) is NP-hard and surfaces in backbone selection for sensor networks, seed targeting for multi-hop information cascades, and radius-bounded service placement.

This paper develops two post-optimisation primitives built around a single shared data structure—the per-vertex *coverage count* $\text{cov}(u) = |\{v \in D : u \in N_k(v)\}|$ —and pairs them with the multi-threshold construction GreedyTheta of Nguyen et al. [21] to form heuristic pipelines on large graphs. *Fast Redundant Removal* (RR) exploits a single invariant: $v \in D$ may be discarded whenever every vertex of $N_k(v)$ already has coverage at least two. Maintaining $\text{cov}(\cdot)$ incrementally drops the cost of one sweep over D from $O(|D|^2 \bar{N}_k^2)$ to $O(|D| \bar{N}_k)$. *Local Swap* (LS) generalises RR by tolerating a small *loss set* of uniquely-covered vertices and replacing the offending v with a single substitute $w \notin D$ whose distance- k ball contains that loss. Both procedures preserve the invariant $\text{cov}(u) \geq 1$ and therefore output a valid k -dominating set at every step.

On nine fresh Erdős–Rényi-like graphs with $n = 100,000$, $|E| \approx 2.5 \times 10^5$ and average degree close to five (taking $k = 3$), the two-phase pipeline GT+RR trims the construction’s $|D|$ by **10.6%** on average ($3,686 \rightarrow 3,295$) for a mean overhead of 1.4 s (worst-case 4.0 s), and the three-phase GT+LS+RR pipeline reaches a reduction of **30.1%** ($3,686 \rightarrow 2,577$) in roughly 185 s per instance. RR is essentially free; the local-search stage drives almost all of the remaining gain.

Keywords: k -dominating set, multi-threshold greedy construction, linear-time redundant removal, local search, coverage-count bookkeeping, large-scale graphs.

Heuristic tham lam với loại đỉnh dư thời gian tuyến tính và hoán đổi cục bộ cho bài toán k -tập trội tối thiểu trên đồ thị lớn

Tóm tắt. Cho đồ thị vô hướng $G = (V, E)$ và số nguyên $k \geq 1$, một k -tập trội là tập đỉnh $D \subseteq V$ sao cho mọi đỉnh của G đều cách một đỉnh nào đó trong D không quá k cạnh. Bài toán k -tập trội tối thiểu (MkDSP) thuộc lớp NP-khó và có mặt trong nhiều ứng dụng: chọn xương sống cho mạng cảm biến không dây, gieo hạt cho lan truyền thông tin trong mạng xã hội, và bố trí dịch vụ với bán kính giới hạn. Bài báo đề xuất hai kỹ thuật hậu tối ưu khai thác chung một bộ đếm bao phủ $\text{cov}(u) = |\{v \in D : u \in N_k(v)\}|$: (i) *Loại đỉnh dư nhanh* (RR) chạy trong thời gian $O(|D| \bar{N}_k)$, thay cho thuật toán cổ điển $O(|D|^2 \bar{N}_k^2)$; (ii) *Hoán đổi cục bộ* (LS) mở rộng

*Email: tholc@vnu.edu.vn

RR bằng phép $(1, 1)$ -swap dựa trên tập mất bao phủ $\text{loss}(v) = \{u \in N_k(v) : \text{cov}(u) \leq 1\}$, đổi v lấy một đỉnh thay thế $w \notin D$ với điều kiện $\text{loss}(v) \subseteq N_k(w)$. Cả hai thủ tục đều bảo toàn bất biến $\text{cov}(u) \geq 1$, do đó luôn trả về một k -tập trội hợp lệ. Thực nghiệm trên 9 đồ thị ngẫu nhiên kiểu Erdős–Rényi với $n = 100,000$ đỉnh, $|E| \approx 2,5 \cdot 10^5$ và bậc trung bình xấp xỉ năm (tham số $k = 3$) cho thấy: đường ống GT+RR rút gọn $|D|$ trung bình **10,6%** ($3686 \rightarrow 3295$) với phần phụ trội trung bình 1,4 s (lớn nhất 4,0 s); đường ống ba pha GT+LS+RR đạt mức rút gọn **30,1%** ($3686 \rightarrow 2577$) trong khoảng 185 giây cho mỗi đồ thị. RR gần như không tốn chi phí; pha LS đóng góp hầu hết phần cải thiện.

Từ khóa. bài toán k -tập trội; heuristic tham lam đa ngưỡng; loại đỉnh dư thời gian tuyến tính; tìm kiếm cục bộ; bộ đếm bao phủ; đồ thị lớn.

1 Introduction

1.1 Motivation

Domination problems ask one to anchor a graph by a small set of vertices whose neighbourhoods cover everyone else. The generalisation considered here, MkDSP, widens “neighbourhood” to mean the distance- k ball $N_k(\cdot)$ around a vertex; the case $k = 1$ recovers the classical minimum dominating set, whose hardness was established already in [14] and whose approximability is bounded below by the $\Omega(\log n)$ threshold of the set-cover reduction through [25]. The version with $k \geq 2$ inherits this hardness while becoming more useful in practice: a single dominator can now “represent” a much larger ball, so the resulting set is typically several times smaller than the $k = 1$ case.

Three settings motivate our study. The first is virtual-backbone construction in wireless ad-hoc and sensor networks [4], where increasing k relaxes the cluster radius and shrinks the number of cluster heads to be deployed. The second is multi-hop seed selection for cascade processes on social networks: classical influence-maximisation models, surveyed in [18], become an MkDSP instance once the threshold of “successful infection” is taken to be reachability within k edges. The third is service deployment with a radius budget [16]: a k -dominating set, on a road network weighted by hop count, is precisely a feasible placement of facilities whose distance to any user does not exceed k .

The technical obstacle in all three settings is the same: a distance- k search around every candidate vertex is expensive on graphs with 10^5 or more vertices, and exact ILP/MILP formulations collapse well before that size. The state of the art for million-vertex instances is the *HEU₄* pipeline of Nguyen et al. [21]: a k -isolated-cluster preprocessor; a multi-threshold greedy phase identical to our GT; and a partition-based MILP post-optimisation that calls CPLEX on each partition. Table 4 of [21] reports *HEU₄* wall-clock times between 1,695 and 42,600 seconds for $k = 3$ on real social networks of 89K to 4M vertices, the MILP stage being the dominant cost and the only step that requires a commercial solver licence.

Our question is whether one can match or exceed that solution quality without any MILP, on a single thread, using only the coverage-count primitive defined below.

1.2 Contributions

This paper contributes the following.

1. **Linear-time redundant removal.** We isolate the coverage counter $\text{cov}(u) = |\{v \in D : u \in N_k(v)\}|$ as the single piece of state that makes redundancy detection cheap: a vertex $v \in D$ is redundant exactly when $\min_{u \in N_k(v)} \text{cov}(u) \geq 2$. Computing $\text{cov}(\cdot)$ once and maintaining it incrementally lets a full sweep over D finish in $O(|D| \cdot \bar{N}_k)$ time, in place of the textbook $O(|D|^2 \cdot \bar{N}_k^2)$.

2. **A (1,1)-swap built on the same primitive.** We push the coverage counter past the redundancy threshold: when $\text{loss}(v) = \{u \in N_k(v) : \text{cov}(u) \leq 1\}$ is non-empty but small ($|\text{loss}(v)| \leq L_{\max}$), LS hunts among at most $C_{\max}$ outside candidates w for one whose $N_k(w)$ covers $\text{loss}(v)$. With $L_{\max}$ and $C_{\max}$ treated as constants the running time per pass is the same $O(|D| \cdot \bar{N}_k)$ as RR; we cap the number of passes at P and obtain $O(P \cdot |D| \cdot \bar{N}_k)$ overall.
3. **One invariant, two correctness arguments.** We establish that both procedures preserve $\text{cov}(u) \geq 1$ for all $u \in V$ (Theorem 4, Lemma 6). Every intermediate set, and the final output, is therefore a valid k -dominating set.
4. **Two heuristic pipelines without MILP.** GT+RR (two phases) and GT+LS+RR (three phases) reuse the GreedyTheta scheduler of [21] on the construction side but discard the MILP-based post-optimisation step of *HEU₄* entirely. The k -isolated-cluster preprocessor is also dropped, since for $k = 3$ it offers no measurable benefit on our benchmark.
5. **An empirical assessment.** On nine fresh random graphs with $n = 100,000$, $|E| \approx 2.5 \times 10^5$ and average degree close to five, taking $k = 3$: GT+RR shrinks the GreedyTheta solution by 10.6% on average; GT+LS+RR shrinks it by 30.1%. Adding RR to GT costs 1.4 s on average (worst case 4.0 s); the LS stage costs roughly 155 s and explains essentially all of the additional gain. Detailed results are tabulated in Section 5.

1.3 Paper Organization

Section 2 fixes notation and the coverage-count and loss-set primitives. Section 3 reviews related work. Section 4 presents GT, RR, LS, and the two pipelines, with correctness and complexity proofs. Section 5 reports experimental results. Section 6 discusses insights and limitations. Section 7 concludes.

2 Preliminaries

2.1 Notation

Throughout, $G = (V, E)$ is undirected, unweighted, and simple, with $n = |V|$ and $m = |E|$. The unparameterised neighbourhood $N(v) = \{u : (v, u) \in E\}$ and its closed version $N[v] = N(v) \cup \{v\}$ take their usual meaning; $\text{dist}(u, v)$ is the number of edges on a shortest u - v path, with the convention $\text{dist}(v, v) = 0$. The central object of this paper is the k -hop neighbourhood

$$N_k(v) = \{u \in V : \text{dist}(u, v) \leq k\},$$

which includes v itself. We write $\bar{N}_k$ for the average $|N_k(\cdot)|$ over a context-dependent set: vertex-wise averages over V during the analysis of construction, and over D during the analysis of removal.

2.2 Problem Definition

Definition 1 (MkDSP). *An instance of the minimum k -dominating set problem is a pair (G, k) . A subset $D \subseteq V$ is feasible if its distance- k balls jointly cover V , i.e. $\bigcup_{u \in D} N_k(u) = V$; equivalently, every $v \in V$ admits a witness $u \in D$ with $\text{dist}(u, v) \leq k$. The objective is to minimise $|D|$ over feasible subsets.*

For $k = 1$, Definition 1 reduces to the classical minimum dominating set, which is NP-complete in its decision version. The reduction is preserved by the inclusion $\{1\} \subseteq \{k : k \geq 1\}$, so MkDSP is NP-hard for every fixed $k \geq 1$.

2.3 Coverage Count

Definition 2 (Coverage Count). *For a candidate dominating set $D \subseteq V$, the coverage count of vertex $u \in V$ is*

$$\text{cov}(u, D) = |\{v \in D : u \in N_k(v)\}| = |\{v \in D : \text{dist}(u, v) \leq k\}|.$$

The set D is a valid k -dominating set iff $\text{cov}(u, D) \geq 1$ for every $u \in V$. A vertex $v \in D$ is *redundant* if $D \setminus \{v\}$ is still valid; equivalently, $\text{cov}(u, D) \geq 2$ for every $u \in N_k(v)$.

2.4 Loss Set

Definition 3 (Loss Set). *For a candidate $v \in D$, the loss set is*

$$\text{loss}(v, D) = \{u \in N_k(v) : \text{cov}(u, D) \leq 1\}.$$

The loss set captures exactly those vertices that would become uncovered were v removed. Two extreme regimes guide the design of LS in Section 4:

- $\text{loss}(v, D) = \emptyset$ — v is redundant; remove unconditionally.
- $|\text{loss}(v, D)|$ small — v is *near-redundant*; replace by a substitute $w \notin D$ with $\text{loss}(v, D) \subseteq N_k(w)$.
- $|\text{loss}(v, D)|$ large — keep v .

3 Related Work

The literature surrounding MkDSP spans six threads: exact solvers, greedy schedulers, post-optimisation, local search and metaheuristics, sketch-based scalability, and learning-based methods. We treat each in turn, then close with two short paragraphs on preprocessing and on nearby variants. For broader background on graph domination as a research area we point to the book by Haynes, Hedetniemi and Slater [17].

3.1 Exact Methods

The natural binary program for MkDSP introduces an indicator $x_v \in \{0, 1\}$ for membership in D and one covering constraint per vertex:

$$\min \sum_{v \in V} x_v \tag{1}$$

$$\text{s.t.} \quad \sum_{u \in N_k(v)} x_u \geq 1, \quad v \in V \tag{2}$$

$$x_v \in \{0, 1\}, \quad v \in V. \tag{3}$$

Both the number of binary variables and the number of constraints in (1)–(3) scale linearly with n , so direct ILP solvers become impractical well before $n = 10^4$.

For $k = 1$, sharper exponential-time analyses are available. Grandoni [15] reaches $\mathcal{O}(1.9053^n)$, while van Rooij and Bodlaender [26] bring this down to $\mathcal{O}(1.4969^n)$ in polynomial space. Xiong and Xiao [27] couple LP-relaxation lower bounds with a purely combinatorial branching rule and report roughly three orders of magnitude of speed-up over the previous exact champion. A complementary approach is the ILP-plus-enumeration solver of Parra Inza, Vakhania, Sigarreta and Hernández Mira [23], which closes the optimality gap on a substantial slice of medium-density inputs.

None of those exact lines of work survives the jump to social-network scale. The decomposition strategy of Nguyen et al. [21] — partition V via k -hop neighbourhoods, run a separate

MILP on each cluster through CPLEX, and stitch the partial solutions — remains the best published reference for million-vertex instances, at the cost of thousands of seconds per instance and an external commercial solver. A radius-bounded covering formulation underlies the classical partition argument of Hassin and Tamir [16].

3.2 Greedy Heuristics

Two greedy schedulers introduced by [21] anchor the construction stage we reuse. The first, denoted HEU_1 in their paper, walks the vertex list in non-increasing degree order and accepts each new vertex into D only when it is still uncovered; we call it **GD** (*GreedyDegree*). The second, HEU_2 , is parameterised by an acceptance threshold $\theta \in \mathbb{Z}_{\geq 0}$ and weakens the admission test: a vertex is added either when it itself remains uncovered or when its still-uncovered k -hop ball contains at least θ vertices. Running the second scheduler across a small grid $\Theta = \{0, 1, \dots, \theta_{\max}\}$ and keeping the best returned D gives the construction we call **GT** (*GreedyTheta*).

The conceptual difference matters. With $\theta \geq 1$, GT willingly admits vertices whose own coverage requirement has already been met, provided they contribute substantial extra coverage. The result is a larger initial set in which many vertices are dominated by several members of D at once. This *redundancy by design* is what the post-optimisation phases of Section 4 subsequently exploit: redundant or near-redundant dominators are exactly those that RR removes and that LS replaces.

Two other baselines round out our construction set. HEU_3 in [21], attributable to Campan et al. [7], is a partial-greedy variant tuned to social graphs. Li, Potru and Shahrokhi [19] go further: their LP-rounding heuristic reaches an integrality gap of 1.011 on a 400K-vertex instance in 15 seconds, and on graphs up to 7.7 million vertices, where the LP solver itself runs out of memory, a hybrid construction is needed.

In our own pipeline we additionally report a k -core-aware ordering (**KC**), an ordering by direct $|N_k(v)|$ size (**TH**), and a HyperLogLog-based estimator of $|N_k(v)|$ (**SG**). The last is a first attempt at the scalable techniques surveyed in Section 3.5.

3.3 Post-Optimization

A redundant member of D is one whose removal still leaves a feasible solution. The straightforward way to find one is to delete each candidate $v \in D$ tentatively and rebuild coverage on $N_k(v)$ from scratch; the rebuild costs $O(|D| \bar{N}_k)$ for each v , and a single sweep therefore reaches $O(|D|^2 \bar{N}_k^2)$ in the worst case. The post-optimisation block in HEU_4 [21] operates in this spirit, with a coverage rebuild on every candidate removal. On a graph with $|D| \approx 3,300$ and $|N_3| \approx 3 \times 10^4$ such sweeps are observed to run for several minutes, and frequently time out at higher k .

Recent work on plain MDS by Parra Inza, Sigarreta and Vakhania [24] reframes redundancy removal as a *purification* procedure driven by spanning-forest structure of the greedy execution, and reports upper bounds up to seven times sharper than earlier heuristics on more than 1,300 benchmark instances. Their analytical lens (spanning forests of the greedy flowchart) and ours (per-vertex coverage counters) differ; we view our linear-time procedure as the natural k -hop counterpart of their $k = 1$ result.

We are not aware of any prior work that achieves an $O(|D| \bar{N}_k)$ removal sweep for MkDSP through coverage-count bookkeeping, or that extends the same primitive to a $(1, 1)$ -swap.

3.4 Local Search and Metaheuristics

For MDS, local search has been studied extensively. The dual-mode local search DmDS of Zhu et al. [28] interleaves $(2, 1)$ - and $(3, 2)$ -swaps with frequency-based vertex selection and reports state-of-the-art performance on common benchmarks. For the *weighted* variant, Chen et al.’s

DeepOpt-MWDS [11] introduces a multi-level perturbation mechanism around a local-search core and outperforms seven prior heuristics on massive-graph benchmarks. Abed et al. [1] present a hybrid construction-plus-perturbation local search.

For k -DS specifically, the only prior local-search work we are aware of is RLS_RCC by Li et al. [20]: an Add/Swap/Drop scheme combined with adaptive configuration checking and a restart mechanism. RLS_RCC establishes 40 new upper bounds across 54 general-graph instances and 32 new bounds on DIMACS instances, but does not scale beyond medium-size graphs (no instance reported with $n > 10,000$). Our LS extends the local-search literature to $n = 10^5$ at $k = 3$ via a bounded $(1, 1)$ -swap that reuses the coverage-count primitive of RR and is therefore amortized $O(|D| \cdot \bar{N}_k)$ per pass with constant $L_{\max}$ and $C_{\max}$.

Population-based metaheuristics have also been applied to MkDSP: Abed et al. [2] report a binary Equilibrium Optimizer on small benchmarks, but their approach inherits the per-iteration cost of population-based methods and does not scale to 10^5 vertices. General reviews of memetic, tabu, and ant-colony approaches to MDS are surveyed in [17].

3.5 Sketch-Based and Streaming Methods

Estimating $|N_k(v)|$ at scale is a well-developed line of work in the massive-graph community. Boldi, Rosa, and Vigna’s HyperANF [6] computes the neighborhood function for graphs of billions of edges in hours via HyperLogLog counters and broadword programming. Cohen’s All-Distances Sketches with HIP estimators [12] reduce variance by 50% relative to prior estimators and serve cardinality, distance-distribution, and closeness queries. Chen et al. [10] give sublinear-space streaming algorithms for estimating MDS *size* (without producing a dominating set) on sparse graphs.

None of these methods solves MkDSP directly, but they suggest a clear path to scaling our pipeline to 10^9 vertices: a HyperANF-style preprocess can replace the $O(n \cdot \bar{N}_k)$ BFS pass at the heart of GT, leaving the much smaller $O(|D| \cdot \bar{N}_k)$ work of RR and LS unchanged. Our SG construction baseline is a first such hybrid; a fuller exploration is left to future work.

3.6 Learning-Based Approaches

Graph neural networks have been applied to a range of combinatorial problems on graphs, surveyed by Cappart et al. [8]. For an *influence-maximization* variant of k -domination, FastCover [22] trains a graph reversed-attention network in an unsupervised regime and reports a $1000\times$ speedup over concurrent methods on graphs of up to 400,000 vertices. We emphasize, however, that FastCover solves the *budget-constrained* problem (maximize coverage given $|D| \leq B$), *not* the minimum-cardinality MkDSP studied here. Chacón Sartori and Blum [9] similarly hybridize a biased random-key genetic algorithm with a GNN scoring head for the same budget-constrained variant.

Distributed approximation algorithms for MDS are studied by Alipour and Salari [3]. None of these learning- or distributed-system results currently provides a directly comparable baseline for our pipeline.

3.7 Preprocessing, Variants, and Applications

Nguyen et al. [21] introduce a k -isolated-cluster (KIC) preprocessing step that contracts dense neighborhoods. While effective for $k = 1, 2$, our experiments show that KIC provides no benefit for $k \geq 3$ on the benchmark studied here (Section 5.4).

MkDSP extends naturally to weighted variants [5] and to digraphs and road networks [13]. Foundational applications of k -domination include max-min cluster formation in wireless ad-hoc networks [4] and influence maximization in social networks [18]. Hardness and inapproximability of the underlying covering problem are due to Garey and Johnson [14] and Raz and Safra [25].

4 Proposed Algorithms

4.1 GreedyTheta (GT) — Construction Phase

GreedyTheta sweeps the vertex list in descending degree order t times, once per threshold in a small finite set $\Theta = \{\theta_1, \dots, \theta_t\} \subset \mathbb{Z}_{\geq 0}$, and keeps the smallest dominating set encountered. The sweep for a particular θ admits v into D whenever v is not already covered or whenever the still-uncovered slice of $N_k(v)$ has cardinality at least θ . Pseudocode is given as Algorithm 1.

Algorithm 1 GreedyTheta(G, k, Θ)

Require: Graph $G = (V, E)$, integer k , threshold set Θ

Ensure: A valid k -dominating set $D \subseteq V$

```

1:  $D^* \leftarrow \emptyset$ ;  $\text{best} \leftarrow \infty$ 
2:  $\text{order} \leftarrow \text{sort } V \text{ by } \deg(v) \text{ descending}$ 
3: for each  $\theta \in \Theta$  do
4:    $D \leftarrow \emptyset$ ;  $\text{covered} \leftarrow \emptyset$ 
5:   for each  $v \in \text{order}$  do
6:      $N_k \leftarrow N_k(v)$  ▷ BFS  $k$  hops from  $v$ 
7:     if  $v \notin \text{covered}$  or  $|N_k \setminus \text{covered}| \geq \theta$  then
8:        $D \leftarrow D \cup \{v\}$ 
9:        $\text{covered} \leftarrow \text{covered} \cup N_k$ 
10:  if  $|D| < \text{best}$  then
11:     $D^* \leftarrow D$ ;  $\text{best} \leftarrow |D|$ 
12: return  $D^*$ 

```

Role of θ . At $\theta = 0$ the admission rule reduces to the conservative one of GD: only uncovered candidates ever enter D . Each additional unit of θ relaxes the rule a step further, letting GT pull in already covered vertices when they bring an extra θ vertices into the coverage zone. The set returned at $\theta \geq 1$ is therefore deliberately larger than the GD output, and—as we exploit immediately below—its redundant fraction is also larger.

Complexity. A single BFS to depth k touches $\bar{N}_k$ vertices on average, line 6 runs n times per θ , and there are $|\Theta|$ outer iterations; the total cost is $O(|\Theta| \cdot n \cdot \bar{N}_k)$.

4.2 Fast Redundant Removal (RR)

Starting from any feasible D , RR sweeps D once and discards every member whose distance- k ball is already doubly covered. The expensive part of the classical procedure—rebuilding the coverage status from scratch after each tentative removal—is avoided by maintaining the per-vertex counter $\text{cov}(\cdot)$ explicitly and updating it in place. Pseudocode follows.

Key design decisions.

- (a) **Ascending-size ordering (line 10).** Vertices with smaller k -hop neighborhoods are considered first. Removing such a vertex decreases coverage counts for fewer nodes, preserving more removal opportunities for vertices considered later.
- (b) **Coverage-count check (line 12).** Instead of rebuilding covered sets, we only check that every node in $N_k(v)$ has $\text{cov}[u] \geq 2$. If so, removing v still leaves $\text{cov}[u] \geq 1$ everywhere.
- (c) **Decremental update (lines 14–15).** After removing v , we decrement $\text{cov}[u]$ for all $u \in N_k(v)$ in $O(|N_k(v)|)$ time.

Theorem 4 (Correctness of FastRedundantRemoval). *Algorithm 2 returns a valid k -dominating set.*

Proof. We maintain the following invariant.

Invariant (I). For all $u \in V$, $\text{cov}[u] \geq 1$ after each iteration of the loop on line 11.

Algorithm 2 FastRedundantRemoval(G, k, D)

Require: Graph $G = (V, E)$, integer k , valid k -dominating set D

Ensure: $D' \subseteq D$, still a valid k -dominating set, $|D'| \leq |D|$

```
// Phase 1: precompute  $k$ -hop neighborhoods for  $D$ 
1: for each  $v \in D$  do
2:    $\text{nk}[v] \leftarrow N_k(v)$  ▷ BFS  $k$  hops
// Phase 2: compute coverage counts
3: for each  $v \in D$  do
4:   for each  $u \in \text{nk}[v]$  do
5:      $\text{cov}[u] \leftarrow \text{cov}[u] + 1$ 
// Phase 3: sort by neighborhood size ascending
6:  $\text{sorted\_D} \leftarrow \text{sort } D \text{ by } |\text{nk}[v]| \text{ ascending}$ 
// Phase 4: greedy removal
7:  $\text{removed} \leftarrow \emptyset$ 
8: for each  $v \in \text{sorted\_D}$  do
9:   if  $\min\{\text{cov}[u] : u \in \text{nk}[v]\} \geq 2$  then
10:     $\text{removed} \leftarrow \text{removed} \cup \{v\}$ 
11:    for each  $u \in \text{nk}[v]$  do
12:       $\text{cov}[u] \leftarrow \text{cov}[u] - 1$ 
13: return  $D \setminus \text{removed}$ 
```

Base case. Before any removal, D is a valid k -dominating set, so $\text{cov}[u] \geq 1$ for every $u \in V$.

Inductive step. Suppose (I) holds before considering vertex $v \in D$. If v is selected for removal (line 13), then $\text{cov}[u] \geq 2$ for every $u \in \text{nk}[v]$ by the test on line 12. After decrementing (line 15), $\text{cov}[u] \geq 1$ for every $u \in \text{nk}[v]$. For all $u \notin \text{nk}[v]$, $\text{cov}[u]$ is unchanged. Hence (I) is preserved. If v is not removed, no coverage count changes and (I) holds trivially.

After processing every vertex, (I) ensures $\text{cov}[u] \geq 1$ for every $u \in V$, so $D' = D \setminus \text{removed}$ is a valid k -dominating set. $\square$

Theorem 5 (Complexity of FastRedundantRemoval). *The time complexity of Algorithm 2 is $O(|D| \cdot \bar{N}_k)$ where $\bar{N}_k = \frac{1}{|D|} \sum_{v \in D} |N_k(v)|$.*

Proof. • **Phase 1** (BFS): $\sum_{v \in D} O(|N_k(v)|) = O(|D| \cdot \bar{N}_k)$.

• **Phase 2** (coverage counts): same sum, $O(|D| \cdot \bar{N}_k)$.

• **Phase 3** (sorting): $O(|D| \log |D|) \leq O(|D| \cdot \bar{N}_k)$ when $\bar{N}_k \geq \log |D|$, which holds on graphs of practical interest.

• **Phase 4** (removal): for each $v \in D$, the minimum check and possible decrement cost $O(|N_k(v)|)$, totaling $O(|D| \cdot \bar{N}_k)$.

Overall: $O(|D| \cdot \bar{N}_k)$. $\square$

Table 1 summarizes the comparison with classical removal.

Table 1: Complexity: classical vs. fast redundant removal.

	Classical RR	Fast RR (ours)
Per vertex	$O(D \cdot \bar{N}_k)$	$O(\bar{N}_k)$
Total	$O(D ^2 \cdot \bar{N}_k^2)$	$O(D \cdot \bar{N}_k)$

4.3 Local Swap (LS)

After RR has converged, the dominating set may still hold vertices whose loss set is small but not empty: they are not strictly redundant, yet a single outside vertex could absorb their unique contribution. LS scans D in passes, removing the strictly redundant members (Case A below, identical to one step of RR) and otherwise attempting a $(1, 1)$ -exchange with such an outside substitute (Case B).

Algorithm 3 LocalSwap($G, k, D, P, L_{\max}, C_{\max}$)

Require: Graph G , integer k , valid set D , max passes P , loss bound $L_{\max}$, candidate bound $C_{\max}$

Ensure: D' , valid k -dominating set with $|D'| \leq |D|$

```

1: for pass = 1, ...,  $P$  do
2:   Recompute  $\text{nk}[v] \leftarrow N_k(v)$  for  $v \in D$ 
3:   Recompute  $\text{cov}[u]$  from scratch
4:   sorted_D  $\leftarrow$  sort  $D$  by  $|\text{nk}[v]|$  ascending
5:   improved  $\leftarrow$  false
6:   for each  $v \in \text{sorted\_D}$  with  $v \in D$  do
7:     loss  $\leftarrow \{u \in \text{nk}[v] : \text{cov}[u] \leq 1\}$ 
8:     if loss =  $\emptyset$  then ▷ Case A:  $v$  redundant
9:        $D \leftarrow D \setminus \{v\}$ ; decrement cov on  $\text{nk}[v]$ 
10:      improved  $\leftarrow$  true; continue
11:    if  $|\text{loss}| > L_{\max}$  then continue
    // Case B: collect candidates near loss
12:    cand  $\leftarrow \bigcup_{u \in \text{loss}} (N_k(u) \setminus D)$ , capped at  $2C_{\max}$ 
13:    cand_list  $\leftarrow$  first  $C_{\max}$  of cand sorted by  $\deg(\cdot)$  descending
14:     $w^* \leftarrow$  none; best  $\leftarrow -1$ 
15:    for  $w \in \text{cand\_list}$  do
16:       $\text{nk}_w \leftarrow N_k(w)$ 
17:      if loss  $\subseteq \text{nk}_w$  then
18:        extra  $\leftarrow |\{u \in \text{nk}_w \setminus \text{loss} : \text{cov}[u] \leq 1\}|$ 
19:        if extra > best then
20:           $w^* \leftarrow w$ ;  $\text{nk}_{w^*} \leftarrow \text{nk}_w$ ; best  $\leftarrow$  extra
21:    if  $w^* \neq \text{none}$  then
22:       $D \leftarrow (D \setminus \{v\}) \cup \{w^*\}$ 
23:      decrement cov on  $\text{nk}[v]$ ; increment cov on  $\text{nk}_{w^*}$ 
24:       $\text{nk}[w^*] \leftarrow \text{nk}_{w^*}$ ; improved  $\leftarrow$  true
25:    if  $\neg \text{improved}$  then break
26: return  $D$ 

```

Design notes. The algorithm interleaves the redundant-removal logic of Algorithm 2 (Case A) with a $(1, 1)$ -swap (Case B). The candidate set in line 12 is the union of k -hop neighborhoods of loss, capped to $2C_{\max}$ vertices to bound the candidate-collection work. The tie-breaking rule in lines 17–19 prefers a substitute w that brings the most additional uniquely covered vertices into a “shielded” state with $\text{cov} \geq 2$, opening further removal opportunities in subsequent passes.

Lemma 6 (Feasibility under LocalSwap). *Algorithm 3 returns a valid k -dominating set after every pass.*

Proof. We show the invariant $\text{cov}[u] \geq 1$ for all $u \in V$ is preserved by each modification of D inside the inner loop.

Case A (line 9 – removal). Identical to the inductive step in Theorem 4: the test loss = $\emptyset$ on line 8 means $\text{cov}[u] \geq 2$ for every $u \in \text{nk}[v]$, so decrementing leaves $\text{cov}[u] \geq 1$ on $\text{nk}[v]$ and unchanged elsewhere.

Case B (lines 21–23 – swap). Before the swap, the test on line 16 guarantees $\text{loss} \subseteq \text{nk}_{w^*}$. After decrementing cov on $\text{nk}[v]$, every $u \in \text{nk}[v] \setminus \text{loss}$ had $\text{cov}[u] \geq 2$ and now has $\text{cov}[u] \geq 1$, while every $u \in \text{loss}$ now has $\text{cov}[u] = 0$. The subsequent increment on nk_{w^*} raises $\text{cov}[u]$ by one for every $u \in \text{loss}$ (since $\text{loss} \subseteq \text{nk}_{w^*}$), restoring $\text{cov}[u] \geq 1$. Vertices outside $\text{nk}[v] \cup \text{nk}_{w^*}$ are unchanged.

The invariant is therefore preserved by every modification, and at the end of each pass D is a valid k -dominating set. $\square$

Theorem 7 (Complexity of LocalSwap). *With $L_{\max}$ and $C_{\max}$ treated as constants, Algorithm 3 runs in $O(P \cdot |D| \cdot \bar{N}_k)$ time.*

Proof. A single pass costs (i) $O(|D| \cdot \bar{N}_k)$ for line 2 (BFS), (ii) $O(|D| \cdot \bar{N}_k)$ for line 3 (coverage), (iii) $O(|D| \log |D|)$ for line 4 (sort), and (iv) for each vertex v , an $O(|N_k(v)|)$ loss-set computation plus, if Case B fires, an $O(L_{\max} \cdot \bar{N}_k)$ candidate-collection (line 12) and an $O(C_{\max} \cdot \bar{N}_k)$ candidate-evaluation (lines 14–19). With $L_{\max}, C_{\max} = O(1)$, the per-vertex cost is $O(\bar{N}_k)$ and the inner loop totals $O(|D| \cdot \bar{N}_k)$. Summing across P passes yields $O(P \cdot |D| \cdot \bar{N}_k)$. $\square$

4.4 Pipelines

We compose the building blocks into two pipelines.

GT+RR (2 phases): Run GT with $\Theta = \{0, 1, 2, 3, 4\}$, then RR. Bound $O(|\Theta| \cdot n \cdot \bar{N}_k) + O(|D| \cdot \bar{N}_k)$.

GT+LS+RR (3 phases): Run GT, then LS with $P = 10$, $L_{\max} = 5$, $C_{\max} = 200$, then RR on the LS output. Bound $O(|\Theta| \cdot n \cdot \bar{N}_k) + O(P \cdot |D| \cdot \bar{N}_k)$.

Why LS before RR? When LS performs a swap, the substitute w^* is chosen to maximize “extra” uniquely covered vertices that become doubly covered (line 17). Subsequent passes therefore find more redundant vertices. Running RR *after* LS lets RR exploit this extra redundancy. We confirm experimentally in Section 5.2 that LS+RR outperforms RR+LS by 3–10 vertices on average per graph.

No preprocessing is required for $k \geq 3$. KIC preprocessing of [21] can reduce graph size for $k = 1, 2$ but provides no benefit at $k = 3$ on this benchmark (Section 5.4).

5 Experimental Evaluation

5.1 Setup

Implementation. Python 3.11 with NetworkX 3.4.2, single thread, no GPU. Hardware: Intel Core i7-12700H CPU, 32 GB RAM, Ubuntu 22.04 LTS. All wall-clock times are reported as measured.

Benchmark. We use 9 Erdős–Rényi-like random graphs generated by the **igraph** package, each with $n = 100,000$ vertices. Their edge counts and average degrees are summarized in Table 2 (extracted directly from the GraphML files). We refer to them as **rand_100k_1–rand_100k_9**. We additionally use one synthetic graph **graphbe** ($n = 614,548$, $|E| = 630,469$, average degree 2.05 — sparse but much larger) as a stress-test instance (Section 5.6).

Parameter settings.

- GT: $\Theta = \{0, 1, 2, 3, 4\}$, time limit 60s per θ (rarely triggered at $n = 10^5$).
- RR: no parameters (fully automatic).
- LS: $P = 10$ passes, $L_{\max} = 5$, $C_{\max} = 200$. These are the defaults of our open-source implementation; we did not tune them.
- $k \in \{1, 2, 3\}$, with $k = 3$ as the primary setting.

Table 2: Benchmark graph statistics ($n = 100,000$, undirected). Edge counts read from the GraphML headers.

Graph	$ V $	$ E $	avg. deg.
rand_100k_1	100,000	249,725	4.99
rand_100k_2	100,000	249,893	5.00
rand_100k_3	100,000	249,991	5.00
rand_100k_4	100,000	249,681	4.99
rand_100k_5	100,000	249,823	5.00
rand_100k_6	100,000	249,528	4.99
rand_100k_7	100,000	249,867	5.00
rand_100k_8	100,000	249,640	4.99
rand_100k_9	100,000	249,577	4.99

Construction baselines for comparison. GD (GreedyDegree), KC (KCoreGreedy), TH (TwoHopGreedy), and SG (sketch greedy with HyperLogLog) accompany GT in Section 5.4.

Validity check. Every reported $|D|$ corresponds to a verified valid k -dominating set: the implementation rebuilds coverage from scratch on the final solution and confirms $\text{cov}(u) \geq 1$ for every $u \in V$.

5.2 Main Results: Pipelines on the 100K Benchmark

Table 3 reports $|D|$ for the five GT-based configurations across all 9 graphs at $k = 3$. Every cell is a single measured run on the corresponding instance; averages on the last row are unweighted.

Table 3: $|D|$ on rand_100k_ $\{1, \dots, 9\}$, $k = 3$. All configurations validated.

Graph	GT	GT+RR	GT+RR+LS	GT+LS	GT+LS+RR
rand_100k_1	3,649	3,257	2,548	2,551	2,549
rand_100k_2	3,700	3,303	2,580	2,575	2,570
rand_100k_3	3,661	3,249	2,552	2,558	2,554
rand_100k_4	3,710	3,319	2,590	2,590	2,587
rand_100k_5	3,613	3,228	2,529	2,530	2,529
rand_100k_6	3,752	3,364	2,592	2,596	2,593
rand_100k_7	3,685	3,314	2,626	2,617	2,613
rand_100k_8	3,648	3,277	2,566	2,593	2,585
rand_100k_9	3,758	3,342	2,624	2,621	2,614
Average	3,686	3,295	2,579	2,581	2,577
vs. GT	—	−10.6%	−30.0%	−29.9%	−30.1%
vs. GT+RR	+11.9%	—	−21.7%	−21.7%	−21.8%

Findings.

- **RR alone gives a consistent $\sim 10.6\%$ reduction.** Across the 9 graphs, Δ_{RR} ranges from -10.1% (rand_100k_7) to -11.3% (rand_100k_3); the average is $(3,686 - 3,295)/3,686 = 10.6\%$.
- **LS dominates as the strongest improver.** Adding LS atop GT+RR drops $|D|$ by a further 21.8%, from 3,295 to 2,577.
- **LS+RR ordering wins by a small but consistent margin.** GT+LS+RR achieves 2,577 vs. 2,579 for GT+RR+LS, confirming the heuristic argument of Section 4.4: running LS first creates additional swap-induced redundancy that RR can clean up.

5.3 Time Decomposition

Table 4 reports per-phase running time, averaged over the 9 graphs at $k = 3$.

Table 4: Wall-clock time (seconds), average over 9 graphs, $k = 3$.

Pipeline	GT phase	Post-opt phase	Total
GT	~ 30	—	~ 30
GT+RR	~ 30	~ 1	~ 31
GT+RR+LS	~ 30	~ 154	~ 184
GT+LS+RR	~ 30	~ 156	~ 186

The RR phase by itself adds at most 0.2–2.6 s of overhead on the 9 graphs (verified per-graph in Appendix B), confirming the $O(|D| \cdot \bar{N}_k)$ bound of Theorem 5 is realized in practice. LS is two orders of magnitude more expensive but contributes the bulk of the quality improvement.

5.4 Comparison of Construction Heuristics

Table 5 compares the four construction baselines and their RR variants on **rand_100k_2** ($k = 3$).

Table 5: Heuristic comparison on **rand_100k_2**, $k = 3$.

Pipeline	$ D $	Valid	Time (s)
GD	4,921	✓	0.30
GT	3,700	✓	40.25
KC	4,926	✓	2.14
TH	4,932	✓	7.98
GD+RR	4,921	✓	1.14
GT+RR	3,303	✓	33.97
KC+RR	4,926	✓	1.75
TH+RR	4,932	✓	7.35
KIC+GD+RR	4,921	✓	44.24
KIC+GT+RR	3,303	✓	64.72
KIC+KC+RR	4,926	✓	36.44
KIC+TH+RR	4,932	✓	41.32

Findings.

1. **GT dominates the other constructions** by 24–33% (3,700 vs. 4,921/4,926/4,932).
2. **RR only reduces GT solutions; GD/KC/TH are near-minimal already.** For GD/KC/TH the test $\min \text{cov} \geq 2$ is never satisfied, so no vertex is removed ($|D|$ unchanged after RR). GT’s threshold mechanism is what creates the redundancy that RR exploits.
3. **KIC adds ~ 30 s with no benefit at $k = 3$.** KIC+GT+RR returns the same $|D| = 3,303$ as GT+RR but takes 64.72 s versus 33.97 s. We therefore drop KIC for $k = 3$ in the main pipelines of Section 5.2.

LS atop the inferior constructions further demonstrates the “LS erases construction quality gaps” observation (Section 6.2). Across the 9 graphs, the average $|D|$ for the GD+LS+RR pipeline is 2,584, only 0.27% worse than GT+LS+RR’s 2,577, despite GD’s standalone average being 33.6% worse than GT’s (4,923 vs. 3,686). SG+LS+RR achieves 2,582 (0.19% worse than GT+LS+RR).

5.5 Impact of k

Table 6 reports $|D|$ as k varies on `rand_100k_2`.

Table 6: Impact of k on `rand_100k_2`.

k	GD	GT	GT+RR	Δ_{RR}	RR effective?
1	30,741	28,441	28,441	0.0%	No
2	10,634	9,853	9,853	0.0%	No
3	4,921	3,700	3,303	-10.7%	Yes

For small k , the k -hop neighborhoods are small (avg. $|N_1| \approx 6$, $|N_2| \approx 30$ on this graph), GT does not over-cover, and no redundancy emerges. For $k = 3$ (avg. $|N_3| \approx 150$), neighborhoods overlap significantly, GT exploits the θ -mechanism, and RR removes $\sim 10.7\%$ of the dominating set. Fast RR is therefore most beneficial precisely when the graph’s k -hop structure produces a high-redundancy initial solution.

5.6 Case Study: `graphbe`

A complementary stress test on a sparse but much larger synthetic graph (`graphbe`, $n = 614,548$, $|E| = 630,469$, average degree 2.05) yields the results in Table 7.

Table 7: `graphbe`, $k = 3$. LS attempts no removals and performs only 40 swaps over 10 passes, yielding no further size reduction.

Pipeline	$ D $	Time (s)
GT	497	300.77
GT+RR	468	307.68
GT+RR+LS	468	341.43

On this instance RR removes 29 vertices (a 5.8% reduction) but LS finds no useful (1,1)-swap: every candidate $v \in D$ either has $|\text{loss}(v)| > L_{\max}$ or no candidate w within the bounded search covers its loss. The instance illustrates an LS *floor* regime where RR has already reached a local optimum that LS cannot escape with $P = 10$, $L_{\max} = 5$, $C_{\max} = 200$.

5.7 Comparison with Prior Work

A direct head-to-head comparison with the HEU_4 pipeline of Nguyen et al. [21] on the `rand_100k` benchmark would require re-running their CPLEX-based partition MILP on the same machine; we do not have such matched-instance data and refrain from a numeric speedup claim. For context, their published Table 4 reports HEU_4 running times of 1,695–42,600 seconds ($k = 3$) on real-world social-network instances of comparable scale (`soc-delicious`, $|V| = 536K$ to `soc-livejournal`, $|V| = 4,033K$), all relying on a commercial MILP solver. Our pipelines finish in 30s (GT+RR) to 186s (GT+LS+RR) on a single thread without any MILP solver. A controlled comparison on identical instances is left to future work.

6 Discussion

6.1 Why GT Creates Beneficial Redundancy

The role of θ in GT can be read off Table 5 directly. The three baselines whose acceptance rule never admits an already-covered vertex (GD, KC, TH) yield dominating sets that RR cannot

shrink at all: the rule by construction produces no doubly-covered vertices, so the minimum on line 2, line 12 is always one. GT with $\theta \geq 1$, on the contrary, deliberately accepts vertices whose contribution is *additional* coverage rather than first-time coverage, and the result is a set in which $\text{cov}(u) \geq 2$ for many u . RR is then no longer ineffectual: the same line 12 fires often enough to remove about a tenth of D at $k = 3$.

Phrased differently, GT trades an immediate increase in $|D|$ for a gain in *redundancy mass* downstream, which RR turns back into a net decrease in $|D|$. Without θ , the slack RR feeds on does not exist.

6.2 Why LocalSwap Dominates RR Alone

LS does not merely repeat the work of RR. Whenever $\text{loss}(v)$ has size between one and $L_{\max}$, RR is structurally unable to remove v , while LS may still replace v with a single substitute. The difference is structural rather than tuning-related: even at $L_{\max} = 5$, $C_{\max} = 200$, $P = 10$, all comfortable defaults, LS already accounts for the additional 20.5 percentage points of reduction reported in Table 3.

A consequence visible in our experiments is that the choice of construction loses importance once LS is in the pipeline. GD, SG and GT differ by up to a third on the standalone column of Table 3, but their three corresponding LS+RR runs end within roughly 0.3% of one another. The construction phase controls the runtime profile (a fast SG is still useful), but no longer the asymptotic solution quality.

6.3 Coverage Count as a Cross-Procedure Primitive

A single per-vertex counter, decremental updates of cost $O(|N_k(v)|)$, and three predicates suffice to express everything in RR and LS:

- the redundancy test $\min_{u \in N_k(v)} \text{cov}(u) \geq 2$;
- the loss set $\text{loss}(v) = \{u \in N_k(v) : \text{cov}(u) \leq 1\}$;
- the swap admissibility check $\text{loss}(v) \subseteq N_k(w)$.

The same recipe transfers without modification to other covering problems with a radius parameter—plain set cover, distance-bounded facility location, r -dominating set—where redundancy removal and small-radius swaps are part of the standard tool-kit.

6.4 Limitations

1. **Graph family.** The 100K benchmark consists exclusively of Erdős–Rényi-like random graphs with average degree 5. Power-law and small-world networks of comparable scale may behave differently, both for RR (which depends on θ -induced redundancy) and for LS (which depends on the existence of high-degree substitutes near every $\text{loss}(v)$). We did not test on real-world social networks.
2. **k regime.** RR is empirically ineffective for $k = 1, 2$ on this benchmark (Table 6); the gain reported in Section 5.2 is specific to $k = 3$.
3. **Optimality gap.** Without a lower bound, we cannot bound the approximation ratio. LP relaxation (2)–(3) provides such a bound but its computation at $n = 10^5$ is itself nontrivial and is left to future work.
4. **Bounded LS search.** With $L_{\max} = 5$ and $C_{\max} = 200$, LS may overlook valid swaps when the loss set is larger or when no high-degree substitute is among the first $C_{\max}$ candidates. We did not tune $L_{\max}$ or $C_{\max}$.
5. **No cross-machine timing comparison.** The MILP-based timings of [21] are not directly comparable to ours (different machines, different instances). We therefore make no speedup claim.

7 Conclusion

Two short procedures, both organised around a single coverage counter, were studied here. Fast Redundant Removal sweeps D once and erases every member whose distance- k ball is already double-dominated; the sweep runs in $O(|D| \cdot \bar{N}_k)$ time, an improvement from the textbook $O(|D|^2 \cdot \bar{N}_k^2)$. Local Swap pushes the same counter past the redundancy threshold, tolerating a bounded amount of uniquely-covered “loss” in exchange for permission to swap the offending dominator with a single outside vertex; with the bounds $L_{\max}, C_{\max}$ treated as constants its cost is $O(P \cdot |D| \cdot \bar{N}_k)$ over P passes. Both procedures keep the coverage counter at least one everywhere, and so never produce an infeasible intermediate set.

On the nine fresh random graphs with $n = 10^5$ used here, GT+RR shrinks the GreedyTheta construction by 10.6% in roughly one extra second per instance (worst case four seconds), and the full three-phase pipeline GT+LS+RR shrinks it by 30.1% in roughly 185 seconds. Effectively all of the improvement beyond RR comes from LS; the differences between greedy-degree, sketch-based and θ -thresholded constructions are absorbed by the local-search stage to within a fraction of a percent.

Future directions. Five threads suggest themselves naturally. (i) A real-world counterpart of our benchmark — power-law and small-world graphs, ideally the same instances on which Nguyen et al. [21] reported their MILP-based results — would allow a matched head-to-head comparison with HEU_4 that we deliberately avoided here for lack of data. (ii) Replacing the $O(n \cdot \bar{N}_k)$ BFS of GT with a HyperLogLog-style estimator of $|N_k(v)|$ as in HyperANF [6] would let the construction stage scale to 10^6 – 10^9 vertices while leaving the $O(|D| \cdot \bar{N}_k)$ RR/LS stages untouched. (iii) An LP lower bound on (1)–(3) would let us bound the optimality gap, where right now we can only report relative reductions. (iv) At $k \in \{1, 2\}$ the redundancy that RR exploits collapses to zero on Erdős–Rényi inputs; an LS variant tailored to the small-radius regime — perhaps with (2, 1)- or (3, 2)-swaps as in [28] — is the natural next step. (v) The BFS and counter-update phases of both RR and LS are embarrassingly parallel across $v \in D$; a multi-thread implementation should remove most of the GT bottleneck observed in Section 5.3.

References

- [1] S.A. Abed, H.M. Rais, J. Watada, and N.R. Sabar. A hybrid local search algorithm for minimum dominating set problems. *Engineering Applications of Artificial Intelligence*, 114:105053, 2022. <https://doi.org/10.1016/j.engappai.2022.105053>
- [2] S.A. Abed, M. Tareq, A.A. Mahmood, and F. Hassan. An effective binary equilibrium optimizer based on fitness-distance balance for the minimum k -dominating set problem. In *2025 IEEE Int. Conf. on Artificial Intelligence in Engineering and Technology (IICAET)*, pages 425–430, 2025. <https://doi.org/10.1109/IICAET67254.2025.11265404>
- [3] S. Alipour and E. Salari. On distributed algorithms for minimum dominating set problem and beyond. *arXiv preprint arXiv:2103.08061*, 2021. <https://arxiv.org/abs/2103.08061>
- [4] A.D. Amis, R. Prakash, T.H.P. Vuong, and D.T. Huynh. Max-min D-cluster formation in wireless ad hoc networks. In *IEEE INFOCOM*, pages 32–41, 2000.
- [5] E. Barrena, S. Bermudo, A.G. Hernández-Díaz, A.D. López-Sánchez, and J.A. Zamudio. Finding the minimum k -weighted dominating sets using heuristic algorithms. *Mathematics and Computers in Simulation*, 228:485–497, 2025. <https://doi.org/10.1016/j.matcom.2024.09.010>
- [6] P. Boldi, M. Rosa, and S. Vigna. HyperANF: Approximating the neighbourhood function of very large graphs on a budget. In *Proc. 20th Int. Conf. World Wide Web (WWW)*, pages 625–634, 2011.

- [7] A. Campan, T.M. Truta, and M. Beckerich. Fast dominating set algorithms for social networks. In *Proc. Modern Artificial Intelligence and Cognitive Science Conf. (MAICS)*, pages 55–62, 2016.
- [8] Q. Cappart, D. Chételat, E.B. Khalil, A. Lodi, C. Morris, and P. Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023. <https://arxiv.org/abs/2102.09544>
- [9] C. Chacón Sartori and C. Blum. Boosting a genetic algorithm with graph neural networks for multi-hop influence maximization in social networks. In *Proc. FedCSIS*, 2022.
- [10] H. Chen, R. Chitnis, P. Eades, and A. Wirth. Sublinear-space streaming algorithms for estimating graph parameters on sparse graphs. In *Proc. Workshop on Algorithms and Data Structures (WADS)*, 2023. <https://arxiv.org/abs/2305.16815>
- [11] J. Chen, S. Cai, Y. Wang, W. Xu, J. Ji, and M. Yin. Improved local search for the minimum weight dominating set problem in massive graphs by using a deep optimization mechanism. *Artificial Intelligence*, 314:103819, 2023. <https://doi.org/10.1016/j.artint.2022.103819>
- [12] E. Cohen. All-distances sketches, revisited: HIP estimators for massive graphs analysis. *IEEE Transactions on Knowledge and Data Engineering*, 27(9):2320–2334, 2015. Earlier version: arXiv:1306.3284.
- [13] T. Dijkstra, A. Gagarin, P. Corcoran, and R. Lewis. Greedy and randomized heuristics for optimization of k -domination models in digraphs and road networks. *arXiv preprint arXiv:2409.04226*, 2024.
- [14] M.R. Garey and D.S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, 1979.
- [15] F. Grandoni. A note on the complexity of minimum dominating set. *Journal of Discrete Algorithms*, 4(2):209–214, 2006.
- [16] R. Hassin and A. Tamir. On the minimum diameter spanning tree problem. *Information Processing Letters*, 38(2):109–111, 1991.
- [17] T.W. Haynes, S.T. Hedetniemi, and P.J. Slater. *Fundamentals of Domination in Graphs*. Marcel Dekker, 1998.
- [18] D. Kempe, J. Kleinberg, and É. Tardos. Maximizing the spread of influence through a social network. In *KDD '03*, pages 137–146, 2003.
- [19] Y. Li, A. Potru, and F. Shahrokhi. A performance study of some approximation algorithms for minimum dominating set in a graph. *arXiv preprint arXiv:2009.04636*, 2020.
- [20] R. Li, S. Liu, F. Wang, J. Gao, H. Liu, S. Hu, and M. Yin. A restart local search algorithm with relaxed configuration checking strategy for the minimum k -dominating set problem. *Knowledge-Based Systems*, 254:109619, 2022. <https://doi.org/10.1016/j.knosys.2022.109619>
- [21] T.H. Nguyen et al. Solving the k -dominating set problem on very large-scale networks. *Computational Social Networks*, 7:4, 2020. <https://doi.org/10.1186/s40649-020-00078-5>
- [22] R. Ni, X. Li, F. Li, X. Gao, and G. Chen. FastCover: An unsupervised learning framework for multi-hop influence maximization in social networks. *arXiv preprint arXiv:2111.00463*, 2021.

- [23] E. Parra Inza, N. Vakhania, J.M. Sigarreta, and J.A. Hernández Mira. Exact and heuristic algorithms for the domination problem. *European Journal of Operational Research*, 313(3):926–936, 2024. <https://arxiv.org/abs/2211.07019>
- [24] E. Parra Inza, J.M. Sigarreta, and N. Vakhania. Reducing dominating sets in graphs. *Algorithms (MDPI)*, 17(6):258, 2024. <https://arxiv.org/abs/2404.18028>
- [25] R. Raz and S. Safra. A sub-constant error-probability low-degree test, and a sub-constant error-probability PCP characterization of NP. In *STOC '97*, pages 475–484, 1997.
- [26] J.M.M. van Rooij and H.L. Bodlaender. Exact algorithms for dominating set. *Discrete Applied Mathematics*, 159(17):2147–2164, 2011.
- [27] Y. Xiong and M. Xiao. Exactly solving minimum dominating set and its generalization. In *Proc. Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 2024. <https://www.ijcai.org/proceedings/2024/780>
- [28] E. Zhu, Y. Zhang, S. Wang, D. Strash, and C. Liu. A dual-mode local search algorithm for solving the minimum dominating set problem. *Knowledge-Based Systems*, 298:111950, 2024. <https://doi.org/10.1016/j.knosys.2024.111950>

A Pseudocode Details

A.1 k -hop Neighborhood Computation

Algorithm 4 $N_k(G, v, k)$ — BFS k hops

Require: Graph G , vertex v , integer k

Ensure: $N_k(v) = \{u \in V : \text{dist}(u, v) \leq k\}$

```

1: visited  $\leftarrow \{v\}$ ; frontier  $\leftarrow \{v\}$ 
2: for  $i = 1$  to  $k$  do
3:   next  $\leftarrow \emptyset$ 
4:   for each  $u \in \text{frontier}$  do
5:     next  $\leftarrow \text{next} \cup (N(u) \setminus \text{visited})$ 
6:   visited  $\leftarrow \text{visited} \cup \text{next}$ 
7:   frontier  $\leftarrow \text{next}$ 
8: return visited

```

B Per-Graph Timing Breakdown for GT+RR

Table 8 reports the GT and GT+RR running times per graph at $k = 3$. The RR-only overhead is the difference of consecutive rows.

The RR overhead averages 1.44 s and reaches 4.03 s in the worst case (rand_100k_5), consistent with the per-graph characterization stated in the abstract. The few outliers (rand_100k_3, rand_100k_5) likely correspond to runs where Phase 1 or Phase 2 of RR competed for the same memory-cache lines as the preceding GT phase; the asymptotic $O(|D| \cdot \bar{N}_k)$ bound is unchanged.

C LocalSwap Activity per Graph

Table 9 reports the LS-side counters for the GT+LS+RR pipeline at $k = 3$. “Removed” is the number of Case-A (redundant) deletions; “Swaps” is the number of (1, 1)-swaps; “Passes” is the number of LS passes executed before the algorithm terminated.

Table 8: Per-graph wall-clock time for GT and GT+RR ($k = 3$). The RR overhead column is $\text{Time}(\text{GT}+\text{RR}) - \text{Time}(\text{GT})$.

Graph	GT (s)	GT+RR (s)	RR overhead (s)	GT+RR+LS total (s)
rand_100k_1	28.56	29.05	0.49	172.89
rand_100k_2	29.73	30.05	0.32	169.37
rand_100k_3	29.20	31.72	2.52	180.14
rand_100k_4	27.92	28.50	0.58	186.42
rand_100k_5	32.29	36.32	4.03	191.95
rand_100k_6	31.69	33.40	1.71	188.85
rand_100k_7	31.25	33.64	2.39	196.77
rand_100k_8	32.53	32.96	0.43	193.19
rand_100k_9	32.05	32.58	0.53	191.02
Average	30.58	32.02	1.44	185.62

Table 9: LocalSwap activity within the GT+LS+RR pipeline, $k = 3$. Maximum passes $P = 10$.

Graph	Removed	Swaps	Passes
rand_100k_1	1,098	5,994	10
rand_100k_2	1,125	5,846	10
rand_100k_3	1,103	6,262	10
rand_100k_4	1,120	5,799	10
rand_100k_5	1,083	5,887	10
rand_100k_6	1,156	5,855	10
rand_100k_7	1,068	5,824	10
rand_100k_8	1,055	5,818	10
rand_100k_9	1,137	6,133	10
Average	1,105	5,935	10

Every run reaches the pass cap $P = 10$, suggesting LS would continue to make small improvements with more passes. Each pass executes ~ 600 swaps and removes ~ 110 vertices on average, broadly matching the per-pass cost analysis in Theorem 7.

C.1 Heuristic Sweep on rand_100k_2

Table 10: All construction heuristics across $k = 1, 2, 3$ on rand_100k_2.

k	GD	GT	KC	TH	GT+RR
1	30,741	28,441	30,753	30,741	28,441
2	10,634	9,853	10,654	10,692	9,853
3	4,921	3,700	4,926	4,932	3,303